Prompt de Implementacion Completo - Agent Swarm

Fecha: 2026-03-21 **Uso:** Copiar este prompt completo a una sesion de Claude Code para ejecutar cada fase.

Prompt Fase 1: Foundation (Escalar a 4 maquinas)

```
Contexto: Estoy escalando el Agent Memory System de 2 a 4 maquinas.

Repo: /Users/casp/sandboxes/superpago/covacha-projects
Sistema existente: scripts/agent_memory/ (config.py, dynamo_client.py,
Tabla DynamoDB: covacha-agent-memory (us-east-1, PAY_PER_REQUEST)

Tareas:
1. Editar scripts/agent_memory/config.py:
   - Agregar MACHINES = ["mac-1", "mac-2", "mac-3", "mac-4"]
   - Agregar MACHINE_SPECIALIZATION dict:
     mac-1: {role: "backend", repos: ["covacha-payment","covacha-core",
     mac-2: {role: "frontend", repos: ["mf-core","mf-payment","mf-marke
     mac-3: {role: "testing", repos: ["*"], labels: ["test","bugfix","e
     mac-4: {role: "ops", repos: ["covacha-notification","covacha-webho

2. Editar .github/workflows/agent_memory_sync.yml:
   - Cambiar cron de */15 a */5 (sync cada 5 min)

3. Editar scripts/agent_memory/dynamo_client.py:
   - Agregar update_heartbeat(team, machine) que actualice last_seen +
   - Agregar check_stale_machines(threshold_seconds=300) que devuelva m

4. Agregar tests en scripts/agent_memory/tests/:
   - test_4_machines_specialization: verifica config de las 4 maquinas
   - test_heartbeat_update: verifica que actualice last_seen
   - test_stale_detection: verifica que detecte maquinas sin heartbeat

5. Ejecutar pytest -v en scripts/agent_memory/tests/ y verificar que pa

Sigue las reglas de CLAUDE.md: type hints, docstrings en espanol, snake
```

Prompt Fase 2: Dependencies + Messages

```
Contexto: Agregando sistema de dependencias y mensajes inter-maquina al

Repo: /Users/casp/sandboxes/superpago/covacha-projects
Sistema existente: scripts/agent_memory/ (ya tiene claim/release/sync/b
Referencia de arquitectura: docs/operational/AGENT-SWARM-ARCHITECTURE.m
```

Tareas:

1. Editar scripts/agent_memory/dynamo_client.py - agregar funciones par
   - add_dependency(blocked_task: str, blocker_task: str) -> None
     PK: DEP#{blocked_task}, SK: DEPENDS_ON#{blocker_task}
   - get_dependencies(task_id: str) -> list[dict]
     Query PK=DEP#{task_id}, devuelve todos los blockers
   - resolve_dependency(blocker_task: str) -> list[str]
     Busca todos DEP# donde blocker=this task, actualiza blocker_status
     Retorna lista de tareas potencialmente desbloqueadas
   - is_task_unblocked(task_id: str) -> bool
     True si todas sus deps tienen blocker_status=done (o no tiene deps

2. Editar scripts/agent_memory/dynamo_client.py - agregar funciones par
   - send_message(from_machine: str, from_team: str, msg_type: str, tas
     PK: MSG#{timestamp}-{from_machine}, SK: BROADCAST
     Incluir ttl=24h, read_by=[]
     Retorna message_id
   - get_unread_messages(machine: str, limit: int = 20) -> list[dict]
     Scan MSG# donde machine NOT IN read_by, ordenar por timestamp desc
   - mark_as_read(message_id: str, machine: str) -> None
     Append machine a read_by[] via list_append

3. Editar scripts/agent_memory/sync_github.py:
   - Al sincronizar issues, parsear labels que contengan "depends-on:"
   - Formato: label "depends-on:43" → add_dependency(this_task, "43")
   - Si el issue body contiene "Depends on #43" o "Blocked by #43" → ig

4. Editar scripts/agent_memory/dynamo_client.py - modificar get_availab
   - Agregar parametro check_dependencies=False
   - Si True, filtrar tareas donde is_task_unblocked()=True

5. Crear scripts/agent_memory/add_dependency.py (CLI):
   - argparse: --task 044 --depends-on 043
   - Llama add_dependency()

6. Crear scripts/agent_memory/check_messages.py (CLI):
   - argparse: --machine mac-2 [--mark-read]
   - Muestra mensajes no leidos
   - Si --mark-read, marca todos como leidos

7. Editar scripts/agent_memory/release_task.py:
   - Despues de release exitoso con status=done, llamar resolve_depende
   - Enviar MSG broadcast "task_completed" automaticamente

8. Tests (OBLIGATORIO):
   - test_add_dependency: agrega dep y la lee correctamente
   - test_dependency_blocking: tarea con dep no resuelta no aparece en
   - test_dependency_resolution: al completar blocker, tarea se desbloq
   - test_circular_dependency_detection: detectar ciclos A->B->A
   - test_send_message: envia y lee mensaje
   - test_mark_as_read: despues de mark, no aparece en unread
   - test_message_ttl: mensajes con ttl expirado no se devuelven

```
9. pytest -v, todos los tests deben pasar.

Reglas: type hints, docstrings espanol, snake_case, max 50 lineas/funci
```

Prompt Fase 3: Contract Registry

```
Contexto: Agregando registro de contratos de API para que backend publi

Repo: /Users/casp/sandboxes/superpago/covacha-projects
Sistema: scripts/agent_memory/

Tareas:

1. Editar scripts/agent_memory/dynamo_client.py - funciones CONTRACT#:
   - publish_contract(name, version, method, path, request_schema, resp
     PK: CONTRACT#{name}-{version}, SK: META
     Retorna contract_id
   - get_contract(contract_id: str) -> dict | None
   - get_contracts_for_task(task_id: str) -> list[dict]
     Busca contratos donde task_id en consumed_by[] o published_by
   - search_contracts(path_pattern: str = None, method: str = None) ->
     Busca contratos por path o method

2. Crear scripts/agent_memory/publish_contract.py (CLI):
   - argparse: --name "SPEI Transfer" --version v1 --method POST --path
     --request-schema '{"clabe":"string"}' --response-schema '{"success
     --published-by mac-1 --task-id 043

3. Crear scripts/agent_memory/contract_detector.py:
   - Funcion detect_flask_contracts(repo_path: str) -> list[dict]
   - Parsea archivos en routes/ buscando @blueprint.route() decorators
   - Extrae method, path, y si hay marshmallow/pydantic schemas los inc
   - Devuelve lista de contratos detectados

4. Editar scripts/agent_memory/release_task.py:
   - Agregar parametro --contract (JSON string)
   - Si se proporciona, llama publish_contract() y envia MSG con contra

5. Editar scripts/agent_memory/bootstrap.py:
   - En CONTEXT.md generado, incluir seccion "Contratos disponibles"
   - Lista contratos relevantes para el modulo/repo de esta sesion

6. Tests:
   - test_publish_contract: publica y lee contrato
   - test_search_contracts_by_path: busca por path pattern
   - test_contract_in_bootstrap: aparece en CONTEXT.md generado
   - test_detect_flask_contracts: detecta rutas de Flask

7. pytest -v, todos pasan.
```

Prompt Fase 4: Agent Loop

```
Contexto: Creando el daemon autonomo que corre en cada maquina para pic

Repo: /Users/casp/sandboxes/superpago/covacha-projects
Sistema: scripts/agent_memory/ (ya tiene deps, msgs, contracts de Fases
Arquitectura: docs/operational/AGENT-SWARM-ARCHITECTURE.md

IMPORTANTE: Este es el componente mas critico. El loop debe ser robusto

Tareas:

1. Crear scripts/agent_memory/agent_loop.py:
   - Clase AgentLoop con metodos:
     - __init__(team, machine, repos, poll_interval=300)
     - run() — loop principal infinito
     - bootstrap_session() — genera CONTEXT.md fresco
     - find_next_task() — respeta deps, especialización, locks
     - claim_and_implement(task) — claim + lanza Claude Code
     - release_and_notify(task, result) — release + msgs + contracts
     - process_messages() — lee y procesa MSG# pendientes
     - heartbeat() — actualiza last_seen

   - Metodo run() pseudocodigo:
     while True:
       try:
         heartbeat()
         process_messages()
         task = find_next_task()
         if not task: sleep(poll_interval); continue
         result = claim_and_implement(task)
         release_and_notify(task, result)
       except Exception as e:
         log_error(e)
         sleep(60)  # backoff

2. Crear scripts/agent_memory/prompt_generator.py:
   - generate_implementation_prompt(task, contracts, learnings, message
   - Genera prompt completo para Claude Code incluyendo:
     - Task description (del issue body)
     - Repo path y branch
     - Contratos de API relevantes
     - Learnings del modulo
     - Mensajes recientes de otras maquinas
     - Quality gates a ejecutar (pytest -v, ruff check, etc.)
     - Instrucciones de commit format y push

3. Crear scripts/agent_memory/heartbeat_checker.py:
   - Puede correr como cron o Lambda
   - check_and_recover_stale(threshold=300)
   - Si maquina stale: release lock, re-encolar task, send MSG
```

```
4. Editar scripts/agent_memory/config.py:
   - AGENT_LOOP_POLL_INTERVAL = 300  # 5 min entre iteraciones
   - HEARTBEAT_INTERVAL = 120  # 2 min entre heartbeats
   - STALE_THRESHOLD = 300  # 5 min sin heartbeat = stale
   - CLAUDE_CODE_MAX_TURNS = 50  # max turns por implementacion
   - CLAUDE_CODE_TIMEOUT = 3600  # 1 hora max por tarea

5. CLI para iniciar loop:
   python scripts/agent_memory/agent_loop.py \
     --team backend --machine mac-1 \
     --repos covacha-payment,covacha-core,covacha-libs

6. Tests:
   - test_find_next_task_respects_deps: no devuelve tareas con deps pen
   - test_find_next_task_respects_specialization: solo tareas de repos
   - test_loop_handles_no_tasks: duerme sin crashear
   - test_loop_handles_claim_collision: reintenta con otra tarea
   - test_heartbeat_recovery: detecta stale y libera
   - test_prompt_generation: genera prompt completo con contracts y lea

7. pytest -v, todos pasan.

Nota: La funcion claim_and_implement() por ahora puede ser un stub que
implementacion. La integracion real con Claude Code --print se hace en
```

Prompt Fase 5: Dispatcher + Integracion Real

```
Contexto: Creando el dispatcher inteligente y conectando el Agent Loop

Repo: /Users/casp/sandboxes/superpago/covacha-projects
Sistema: scripts/agent_memory/ (Fases 1-4 completas)

Tareas:

1. Crear scripts/agent_memory/dispatcher.py:
   - Clase TaskDispatcher:
     - evaluate_board() — lee todas las tareas + maquinas
     - score_assignment(task, machine) -> float
       Score = (urgency * 0.3) + (specialization_match * 0.4) + (deps_r
     - assign_tasks() — asigna tareas a maquinas idle
     - rebalance() — si maquina bloqueada >15 min, re-asigna

   - Scoring:
     - urgency: P1=1.0, P2=0.7, P3=0.4
     - specialization_match: 1.0 si labels match machine, 0.3 si no
     - deps_resolved: 1.0 si todas, 0.0 si alguna pendiente
     - machine_idle_time: normalizado, mas tiempo idle = mas score

2. Integrar Agent Loop con Claude Code:
   - En agent_loop.py, implementar claim_and_implement() real:
     - Genera prompt via prompt_generator.py
```

```
     - Ejecuta: claude --print --model {model} -p "{prompt}" --max-turn
     - Parsea output para detectar: tests passed, files changed, commit
     - Retorna result dict con status, learnings, contract (si aplica)

3. Crear .github/workflows/dispatcher.yml:
   - Trigger: schedule (cada 10 min) o workflow_dispatch
   - Ejecuta dispatcher.py
   - Envia MSG# con asignaciones

4. Crear scripts/agent_memory/team_dashboard.py:
   - Genera HTML simple con:
     - Estado de las 4 maquinas (idle/working/stale)
     - Tarea actual de cada una
     - Tareas disponibles con deps
     - Mensajes recientes
     - Learnings acumulados
   - Opcion: subir a S3 como pagina estatica

5. Tests:
   - test_score_backend_task_on_backend_machine: score alto
   - test_score_backend_task_on_frontend_machine: score bajo
   - test_rebalance_stale: re-asigna tarea de maquina stale
   - test_dispatcher_no_double_assign: no asigna misma tarea a 2 maquin

6. pytest -v, todos pasan.
```

Prompt para instalar Context Mode en cada maquina

```
Instala Context Mode como plugin de Claude Code en esta maquina.

Pasos:
1. Ejecutar: /plugin marketplace add mksglu/context-mode
2. Ejecutar: /plugin install context-mode@context-mode
3. Reiniciar Claude Code
4. Verificar: /context-mode:ctx-doctor — todos los checks deben ser [x]
5. Verificar: /context-mode:ctx-stats — debe mostrar 0 savings (sesion

Si tambien se usa Cursor en esta maquina:
6. npm install -g context-mode
7. Crear .cursor/mcp.json con: {"mcpServers":{"context-mode":{"command"
8. Crear .cursor/hooks.json con hooks de preToolUse y postToolUse
9. cp node_modules/context-mode/configs/cursor/context-mode.mdc .cursor
```

Orden de ejecucion

```
Mac-1 (lead):    Fase 1 → Fase 2 → Fase 3 → commit + push cada fase
Mac-2:           Instalar Context Mode (mientras mac-1 hace Fases 1-3)
Mac-3:           Instalar Context Mode (mientras mac-1 hace Fases 1-3)
Mac-4:           Instalar Context Mode (mientras mac-1 hace Fases 1-3)
```

Mac-1:	Fase 4 (Agent Loop)
Mac-3:	Tests de Fases 1-3 (QA)
Mac-1:	Fase 5 (Dispatcher)
Mac-2,3,4:	Probar Agent Loop con tareas de prueba
Todas:	Go live — Agent Loop corriendo en las 4 maquinas